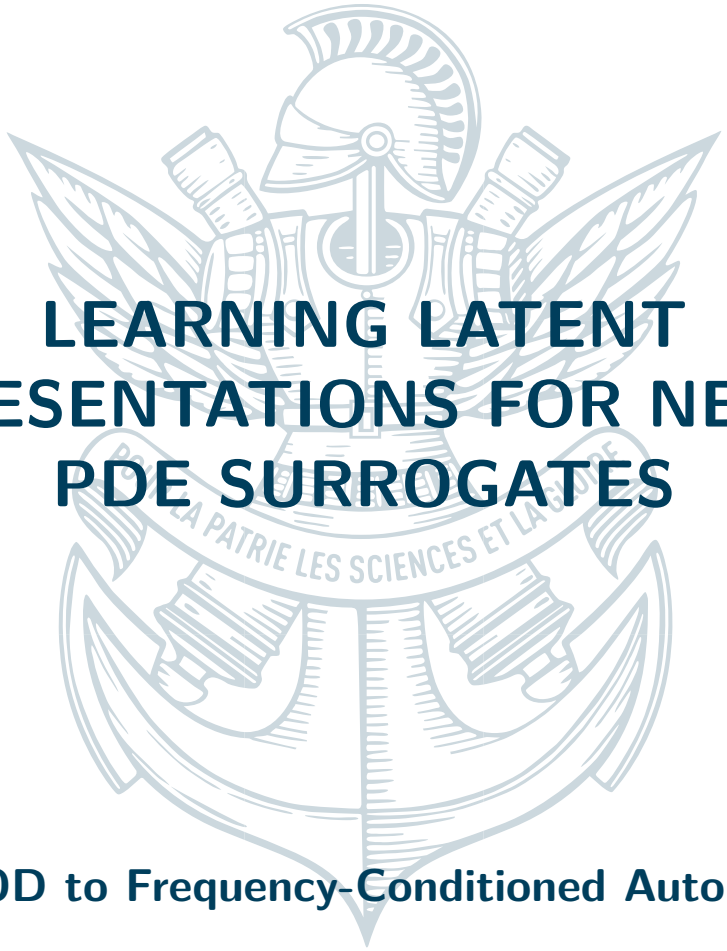


LEARNING LATENT REPRESENTATIONS FOR NEURAL PDE SURROGATES

From POD to Frequency-Conditioned Autoencoders

April 14, 2026

Keyvan Attarian

CONTENTS

1 INTRODUCTION

The numerical simulation of physical systems governed by partial differential equations (PDEs) underpins a vast range of scientific and engineering disciplines, from computational fluid dynamics and structural mechanics to atmospheric modelling and chemical process engineering. High-fidelity solvers—whether based on finite elements, finite volumes, or spectral methods—can capture complex multi-scale phenomena with great accuracy, but at a computational cost that renders them impractical for applications requiring thousands or millions of forward evaluations. Real-time decision making, Bayesian inference over large parameter spaces, optimal experimental design, and digital twin frameworks all fall into this category: each query to the solver may take minutes to hours, making exhaustive exploration computationally intractable [1, 7].

This tension between accuracy and computational cost has motivated a rapidly growing body of work on *surrogate modelling* (also referred to as emulation or metamodelling), in which a cheaper approximate model replaces the high-fidelity solver for a given parameter range. Classical surrogate approaches, such as Gaussian processes or polynomial chaos expansions, offer rigorous uncertainty estimates but scale poorly with input dimensionality and struggle to capture the high-dimensional structure of field-valued outputs. Data-driven methods based on deep learning have emerged as a compelling alternative, offering both the flexibility to represent complex non-linear maps and the ability to produce full spatial fields as outputs [4, 7].

A particularly productive paradigm in this context is *physics-informed machine learning*, which seeks to embed physical knowledge—whether through governing equations, symmetry constraints, or conservation laws—directly into the learning architecture or loss function [4, 5]. The present work draws on this philosophy by leveraging the structure of the Laplace transform as a physically motivated temporal basis, decomposing transient fields into independent frequency components that are individually amenable to surrogate modelling.

Application context. The motivating application is the emulation of atmospheric dispersion models for pollutant concentration prediction, including methane (CH4) [2, 3]. Replacing such simulations with fast neural surrogates accelerates real-time monitoring and inverse source identification.

Central contribution. The high dimensionality of field-valued outputs—up to $N_t \times N^2$ degrees of freedom per simulation—makes direct regression intractable. The central approach is to first learn a compact latent representation of the solution fields, either through linear dimensionality reduction (POD, SVD) or through non-linear autoencoders, and then train a surrogate that maps physical parameters $\boldsymbol{\theta}$ into this latent space. This decoupling reduces the surrogate output dimensionality by orders of magnitude while allowing each component to be trained and validated independently.

Outline. Section 2 introduces the framework on a stationary 2-D convection-diffusion problem. Section 3 presents the transient CH4 problem, comparing linear SVD-based latent representations against a non-linear frequency-conditioned autoencoder, followed by the full surrogate pipeline and a residual correction post-processor.

2 PRELIMINARY STUDY: STATIONARY DATASET

2.1 Problem Statement and Dataset

The stationary test case is a convection-diffusion problem on the unit square $\Omega = [0, 1]^2$:

$$-D\,\Delta U + \boldsymbol{b} \cdot \nabla U = f \quad \text{in } \Omega, \qquad U = 0 \quad \text{on } \partial\Omega, \tag{1}$$

where $D > 0$ is the diffusivity, $\boldsymbol{b} = (b_x, b_y)$ is the convection velocity, and f is a point-source intensity. The parameter vector $\boldsymbol{\theta} = (D, b_x, b_y, f) \in \mathbb{R}^4$ fully specifies a solution instance, and the surrogate task is to learn the forward map $\boldsymbol{\theta} \mapsto U \in \mathbb{R}^{N \times N}$.

The dataset is generated by uniformly sampling $\boldsymbol{\theta}$ over the parameter domain and solving (1) using P1 finite elements with SUPG/GLS stabilisation (FEniCS/DOLFINx). Solutions are interpolated onto a uniform $N \times N$ grid. For machine learning, each field U is centred and rescaled to $[-1, 1]$ using per-pixel statistics computed on the training set; the parameters $\boldsymbol{\theta}$ are standardised using pre-computed mean and standard deviation.

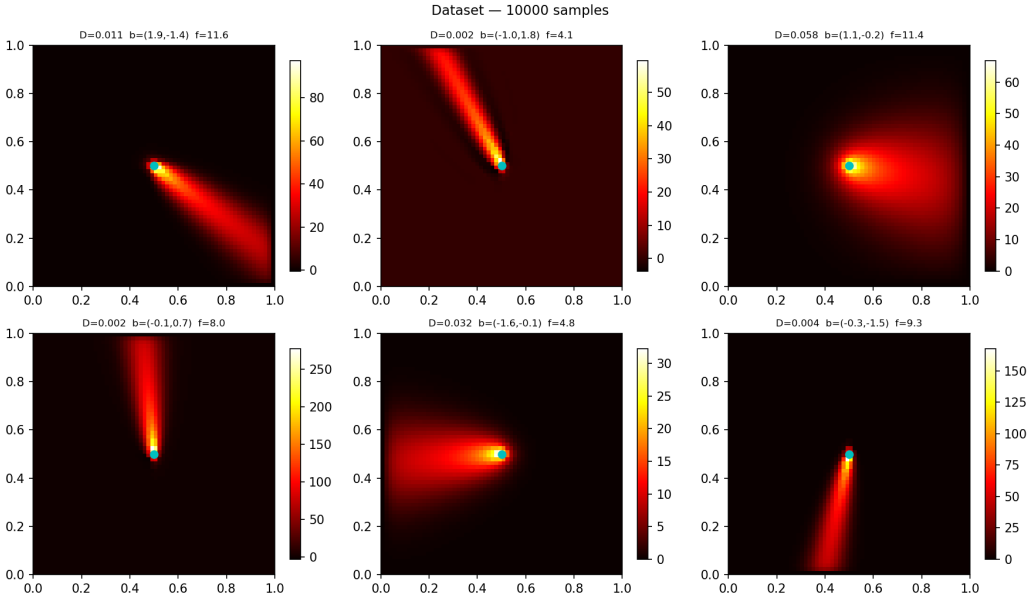


Figure 1: Representative sample of the generated dataset.

2.1.1 • Stage 1 — Variational Autoencoder

The first stage trains a variational autoencoder to compress solution fields U into a low-dimensional latent code $z \in \mathbb{R}^{d_z}$ ($d_z = 32$). The encoder is a strided convolutional network that maps U to the parameters (μ, σ^2) of a Gaussian approximate posterior $q(z|U) = \mathcal{N}(\mu, \sigma^2 I)$. The decoder then reconstructs an estimate $\hat{U}$ from samples of z via transposed convolutions. The training objective combines reconstruction fidelity with distributional regularisation:

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(z|U)}\left[\|U - \hat{U}\|^2 + \lambda_\nabla \|\nabla U - \nabla \hat{U}\|^2\right] + \beta\, \mathrm{KL}\big(q(z|U)\,\|\,\mathcal{N}(0, I)\big), \tag{2}$$

where the expectation is taken over the posterior, the second term inside penalises spatial discontinuities, and the KL divergence encourages the latent distribution to remain close to a unit Gaussian. A free-bits threshold λ_{fb} is applied per latent dimension to prevent posterior collapse.

2.1.2 • Stage 2 — Indirect Decoder $\boldsymbol{\theta} \to z \to U$

The second stage decouples surrogate training from representation learning by freezing the VAE decoder. A two-layer MLP is trained to regress from normalised parameters $\boldsymbol{\theta}_{\text{norm}}$ to the latent code z. At test time, the predicted code is then decoded by the frozen VAE decoder to recover the full field estimate. The surrogate loss combines MSE reconstruction error on the decoded field with the spatial gradient penalty already present in (2), ensuring consistency with the representation learned by the VAE.

2.2 Results

3
TRANSIENT CH4 CONCENTRATION FIELDS

3.1 Dataset and Temporal Representation

3.1.1 • Dataset

The transient experiments use a dataset of 8 100 simulations of atmospheric methane concentration fields. Each simulation yields a time series of 150 snapshots on a 200×200 spatial grid, forming a tensor of shape $(8100, 150, 200, 200)$. The physical parameter vector is $\boldsymbol{\theta} = (k, A, C)$, representing the reaction rate, source amplitude, and initial concentration respectively. To accommodate the ~ 17 GB total dataset size, all data are loaded and processed using memory-mapped arrays, avoiding materialisation of the full tensor in RAM at any point.

3.1.2 • Laplace Transform

Rather than operating in physical time, the surrogate works in the Laplace frequency domain. The Laplace transform of a time-dependent field is approximated via a discretised Bromwich contour integral, computed efficiently using the discrete Fourier transform with exponential damping:

$$\hat{U}(s_k) = \int_0^T U(t)\, e^{-s_k t}\, \mathrm{d}t \approx \mathrm{DFT}\big[U(t) \cdot w_t \cdot e^{-\gamma t}\big], \qquad s_k = \gamma + i\omega_k, \tag{3}$$

where w_t are trapezoidal integration weights and $\gamma \geq 0$ is a damping parameter that improves numerical stability. The spatial field U is transformed at each frequency s_k into a complex-valued field $\hat{U}(s_k) \in \mathbb{C}^{N \times N}$. For real-valued physical fields, Hermitian symmetry provides $\hat{U}(s_{N_t - k}) = \overline{\hat{U}(s_k)}$, so only $N_{\text{half}} = N_t/2 + 1$ frequencies need be stored. In practice, each frequency s_k is represented as the pair of real fields $(\Re(\hat{U}_k), \Im(\hat{U}_k)) \in \mathbb{R}^{2 \times N \times N}$.

The surrogate models only the first $k_{\max}$ frequencies (specifically, $k_{\max} = 20$), which typically capture the essential dynamics while reducing computational cost. Higher frequencies ($k > k_{\max}$) are set to zero in the normalised representation, corresponding to the mean field after denormalisation. Inversion to physical time is performed by discrete Fourier inversion with exponential compensation on the full reconstructed spectrum.

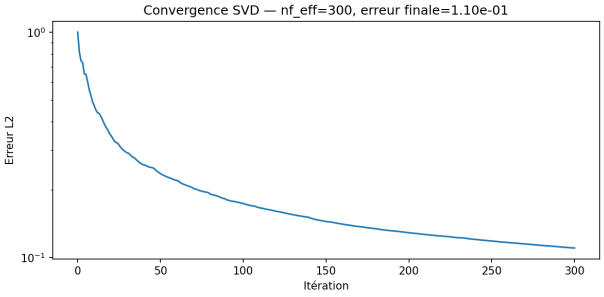
3.2 Latent Space Representation

3.2.1 • SVD-Based Approaches

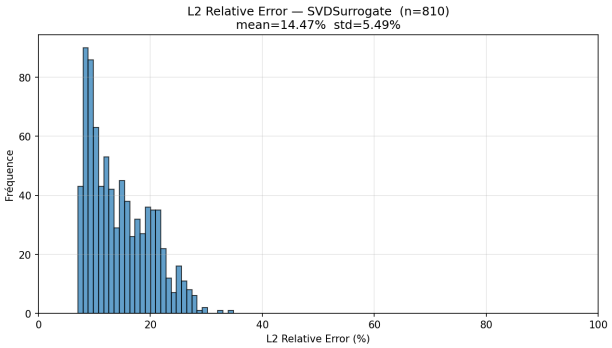
Tucker POD. The Tucker POD approach [1] decomposes the transient field tensor via a low-rank factorisation. The spatially sub-sampled training tensor $\boldsymbol{H} \in \mathbb{R}^{n_r \times n_s \times N_t}$ (with n_r and n_s being the subsampled spatial dimensions and N_t the number of time steps) is factorised into a sum of separable modes:

$$H_{r,s,t} \approx \sum_{j=1}^{n_f} \alpha_j\, F_{r,j}\, G_{s,j}\, P_{t,j}, \tag{4}$$

The factors $\boldsymbol{F}$, $\boldsymbol{P}$, and singular values $\boldsymbol{\alpha}$ are computed once and held fixed across all test samples. The simulation-dependent coefficients $\boldsymbol{G} \in \mathbb{R}^{n_s \times n_f}$ encode information about the individual solutions. A four-layer MLP is trained to predict these coefficients from the physical parameters: $\boldsymbol{\theta} \mapsto \boldsymbol{g} \in \mathbb{R}^{n_f}$. Reconstruction of a full field follows by applying the fixed spatial and temporal modes to the predicted coefficients.



(a) Reconstruction error vs. number of retained modes n_f.



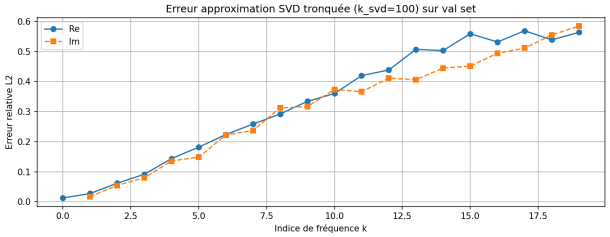
(b) Relative L^2 error on the test set.

Figure 2: Tucker POD surrogate.

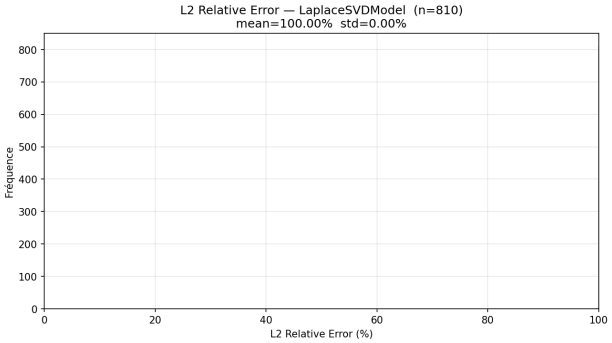
Per-frequency SVD. An alternative linear approach applies truncated SVD independently to each Laplace frequency. For frequency k, the real and imaginary parts of $\hat{U}_k$ across all training samples are arranged into a matrix, and the leading k_{svd} right singular vectors $\boldsymbol{V}^{(k,r)}$ and $\boldsymbol{V}^{(k,i)}$ are extracted:

$$\Re(\hat{U}_k) \approx \boldsymbol{c}^{(k,r)} \big(\boldsymbol{V}^{(k,r)}\big)^\top, \qquad \Im(\hat{U}_k) \approx \boldsymbol{c}^{(k,i)} \big(\boldsymbol{V}^{(k,i)}\big)^\top.$$

The spatial basis $\boldsymbol{V}$ is fixed during training. For each frequency k, a dedicated three-layer MLP learns to predict the normalised coefficients $\boldsymbol{c}^{(k,r)}$ and $\boldsymbol{c}^{(k,i)}$ from the physical parameters. Field reconstruction follows by projecting the predicted coefficients onto the fixed bases; the full time series is then recovered by inverting the Laplace transform as in (3).



(a) SVD reconstruction error vs. k_{svd}.



(b) Relative L^2 error on the test set.

Figure 3: Per-frequency SVD surrogate in the Laplace domain.

Both SVD approaches constrain the latent representation to a fixed linear subspace. When the Laplace fields vary non-linearly across the parameter space, this becomes a binding limitation.

3.2.2 • Autoencoder-Based Latent Space

Both SVD approaches above constrain the surrogate to lie in a fixed linear subspace extracted from the training data. When Laplace fields exhibit non-linear variation across the parameter space, this linearity becomes restrictive. To overcome this limitation, a learned autoencoder is trained to extract a non-linear latent representation that is shared across all frequencies. The encoder and decoder are conditioned on the frequency index k, allowing a single pair of networks to process fields at any frequency while adapting their behaviour through frequency-specific control signals.

Sinusoidal frequency encoding. The frequency index k is first normalised to the range $[0, 1]$ by dividing by N_{half}, giving a frequency ratio $f = k/N_{\text{half}}$. This scalar is then embedded into a higher-dimensional vector $\boldsymbol{e}(f) \in \mathbb{R}^{64}$ via sinusoidal basis functions inspired by positional encoding in transformers. The embedding is computed as follows: for each octave $i = 0, 1, \ldots, L-1$, sinusoidal components $\sin(2^i \pi f)$ and $\cos(2^i \pi f)$ are computed, yielding a $2L$-dimensional vector. This vector is passed through a small MLP to produce the final frequency embedding:

$$\boldsymbol{e}(f) = \text{MLP}\Big(\big[\sin(2^i \pi f),\ \cos(2^i \pi f)\big]_{i=0}^{L-1}\Big), \qquad L = 8. \tag{5}$$

This multi-scale representation allows the network to capture frequency information at multiple resolutions simultaneously, enabling fine discrimination among different frequency bands.

FiLM conditioning. To allow the encoder and decoder weights to be shared across all frequencies while adapting their behaviour to each specific frequency, the Feature-wise Linear Modulation (FiLM) mechanism is employed. At each convolutional layer ℓ of the encoder and decoder, the frequency embedding $\boldsymbol{e}(f)$ is passed through two learned linear projections to produce a channel-wise gain vector $\gamma_\ell(\boldsymbol{e}(f)) \in \mathbb{R}^{C_\ell}$ and a channel-wise bias vector $\beta_\ell(\boldsymbol{e}(f)) \in \mathbb{R}^{C_\ell}$, where C_ℓ denotes the number of feature channels at layer ℓ. These gains and biases are used to modulate the intermediate activations $\boldsymbol{x}_\ell$ element-wise:

$$\boldsymbol{x}_\ell \ \leftarrow\ \boldsymbol{x}_\ell \odot \big(1 + \gamma_\ell(\boldsymbol{e}(f))\big) \ +\ \beta_\ell(\boldsymbol{e}(f)). \tag{6}$$

For each channel c, the gain $\gamma_\ell^{(c)}$ applies a multiplicative correction and the bias $\beta_\ell^{(c)}$ applies an additive correction, both functions of the frequency alone. This design allows all frequencies to share the same convolutional weights while the FiLM parameters (derived from the frequency embedding) provide frequency-specific adjustment at each layer.

Architecture and training. The autoencoder encoder takes a two-channel input $(\Re(\hat{U}_k), \Im(\hat{U}_k)) \in \mathbb{R}^{2\times N\times N}$ representing the real and imaginary parts of a Laplace field at frequency k, and progressively downsamples it through strided convolutions to produce a latent code $z \in \mathbb{R}^{d_z}$. Each convolutional block is followed by FiLM modulation using the frequency embedding $\boldsymbol{e}(f)$. The decoder reverses this process, starting from z and upsampling through transposed convolutions, also FiLM-conditioned at each layer. The decoder outputs two separate branches, one predicting $\Re(\hat{U}_k)$ and the other $\Im(\hat{U}_k)$.

Training operates on the Cartesian product of all (simulation, frequency) pairs in the dataset, with the objective:

$$\mathcal{L}_{\text{AE}} = \|\hat{U} - U\|^2 + \beta\, \|\hat{U}\|^2, \tag{7}$$

where the first term is the reconstruction MSE and the second is an L^2 regularisation penalty on the output. To ensure diverse frequency coverage within each mini-batch, (simulation, frequency) pairs are reshuffled at the start of each epoch.

3.3 Surrogate and Fine-Tuning

3.3.1 • Per-Frequency Surrogate $\boldsymbol{\theta} \to z$

Once the frequency-conditioned autoencoder has been trained, its decoder is frozen, and independent surrogates are trained for each frequency. The surrogate operates only on the first $k_{\max}$ frequencies (with $k_{\max} = 20$); higher frequencies are not explicitly predicted. For frequencies $k = 0, 1, \ldots, k_{\max}$, a four-layer MLP with layer normalisation learns the mapping from normalised physical parameters to the latent code: $\boldsymbol{\theta}_{\text{norm}} \mapsto z_k \in \mathbb{R}^{d_z}$. The shared decoder is kept frozen to preserve the learned latent structure.

At inference time, each predicted code z_k is decoded using the frozen decoder, which is itself conditioned on the frequency ratio f_k via FiLM, yielding predictions for frequencies $k \le k_{\max}$. Frequencies $k > k_{\max}$ are left at zero in the normalised space, which corresponds to the mean value after denormalisation. The complete Laplace spectrum is then obtained by Hermitian symmetry (the upper frequencies are conjugates of the lower ones), and the time-domain field is recovered by Laplace inversion as in (3).

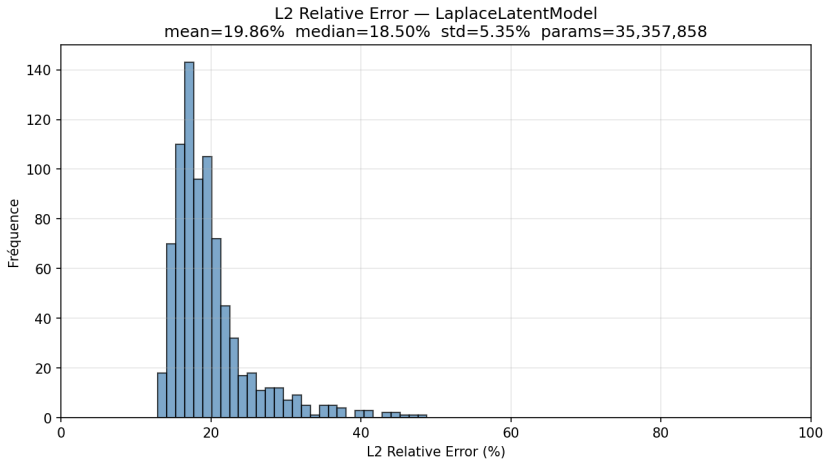


Figure 4: Relative L^2 error distribution for the baseline latent surrogate (mean: 19.86%, median: 18.50%).

3.3.2 • End-to-End Fine-Tuning

To further improve accuracy, the frozen decoder is unfrozen and the entire model—both the per-frequency MLPs and the shared decoder—is fine-tuned jointly end-to-end on the training set. Differential learning rates are employed to balance the adaptation: $\eta_{\text{surr}} = 5 \times 10^{-5}$ for the per-frequency parameter-to-latent MLPs and $\eta_{\text{dec}} = 10^{-5}$ for the decoder. The training batches are organised in a frequency-first fashion, processing all simulations for a single frequency in one mini-batch. This structure allows for efficient batching: a single forward call through the FiLM-conditioned decoder handles all batch samples simultaneously, improving computational efficiency compared to a naive per-simulation approach.

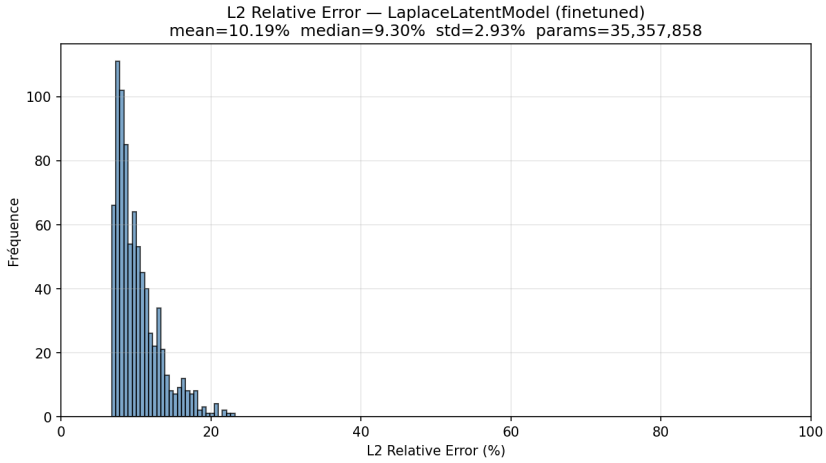


Figure 5: Relative L^2 error distribution after end-to-end fine-tuning (mean: 10.19%, median: 9.30%). Fine-tuning reduces error by approximately 48% compared to the frozen baseline.

3.4 Post-Processing and Results

3.4.1 • Residual Correction

A UNet residual corrector is applied post hoc, frame by frame, to the surrogate output. A UNet is an encoder–decoder convolutional network in which the encoder progressively downsamples the spatial resolution through successive convolution and pooling operations, while the decoder restores the original resolution through transposed convolutions. At each resolution level, the decoder receives, via a skip connection, the feature maps produced by the encoder at the corresponding level; these are concatenated channel-wise before the next convolution. This architecture allows the network to combine coarse, context-rich features from the bottleneck with fine spatial detail preserved by the skip connections. A standard autoencoder forces all information through the bottleneck, discarding spatial detail that cannot be encoded in the latent code; the skip connections of the UNet bypass this bottleneck, providing the decoder with direct access to high-resolution encoder features and enabling precise spatial localisation in the output.

Given the surrogate prediction $U_{\text{pred}}(t) \in \mathbb{R}^{N\times N}$, the residual correction operates as follows. The predicted field is first normalised per-frame using its own spatial statistics (mean μ and standard deviation σ), then passed through a UNet that predicts a residual correction in the normalised space. The residual is rescaled back to physical space and added to the original prediction:

$$U_{\text{corr}} = U_{\text{pred}} + \sigma(U_{\text{pred}}) \cdot r\left(\frac{U_{\text{pred}} - \mu(U_{\text{pred}})}{\sigma(U_{\text{pred}})}\right), \tag{8}$$

Here $r(\cdot)$ is a three-level UNet with skip connections, employing $c = 16$ base channels and approximately 481k parameters. A crucial design choice is that the final output layer of the UNet is zero-initialised, ensuring that $r \equiv 0$ at the start of training. This initialisation strategy causes the corrected field to equal the surrogate prediction initially, allowing the network to learn only the correction needed rather than the full field.

3.4.2 • Training the Corrector

A key practical consideration in training the residual corrector is computational efficiency. Rather than calling the frozen surrogate model repeatedly during training, all $(U_{\text{pred}}, U_{\text{true}})$ pairs are pre-computed and stored on disk as memory-mapped arrays before the corrector training begins. This pre-computation step ensures that no forward pass through the surrogate is needed during the training loop, significantly accelerating convergence.

The corrector loss combines reconstruction accuracy with spatial smoothness. Denoting $r_{\text{pred}} = U_{\text{corr}} - U_{\text{pred}}$ as the predicted residual and $r_{\text{true}} = U_{\text{true}} - U_{\text{pred}}$ as the target residual, the loss function is:

$$\mathcal{L}_{\text{corr}} = \left\| \frac{r_{\text{pred}} - r_{\text{true}}}{\sigma_r} \right\|^2 + \lambda_\nabla \left\| \frac{\nabla r_{\text{pred}} - \nabla r_{\text{true}}}{\sigma_r} \right\|^2, \tag{9}$$

where $\sigma_r = \sigma(r_{\text{true}})$ is the standard deviation of the target residual field, and $\lambda_\nabla = 10$ is the gradient weighting parameter. Both the residual MSE and the spatial gradient MSE are normalised by σ_r, which accounts for the magnitude of the true correction and prevents the loss from being dominated by frames requiring large corrections.

3.4.3 • Results and Comparison

The residual correction stage applies the UNet post-processor to the finetuned surrogate output, frame by frame. The error distribution is shown in Figure 6.

Key findings:

- **Fine-tuning impact.** End-to-end fine-tuning of the per-frequency MLPs and shared decoder reduces median error from 18.50% to 9.30%, a gain of approximately 50%. The reduced standard deviation (5.35% $\rightarrow$ 2.93%) indicates that fine-tuning not only improves mean performance but also makes predictions more consistent across the test set.

- **Residual correction.** The UNet residual corrector achieves median error of 4.46%, representing a further 52% reduction relative to the finetuned model. The error range tightens dramatically: from

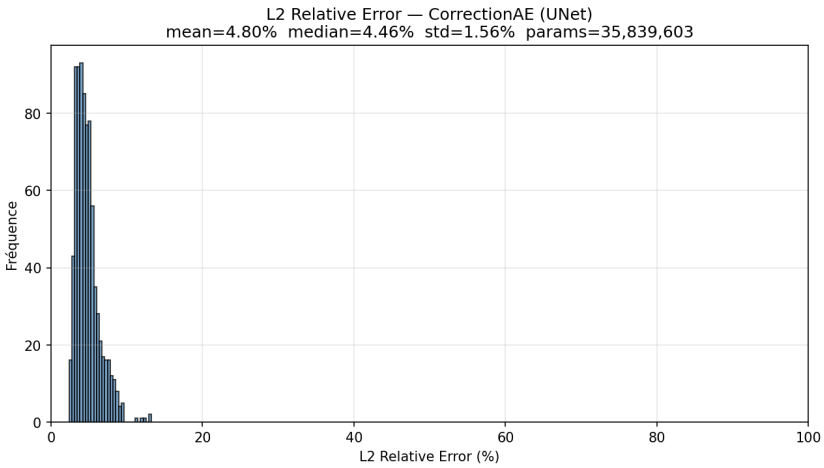


Figure 6: Relative L^2 error distribution after residual UNet correction (mean: 4.80%, median: 4.46%). The correction reduces error by approximately 76% compared to the baseline latent surrogate.

Method	Parameters	Mean (%)	Median (%)	Std (%)	Range (%)
LaplaceLatentModel (baseline)	35.36 M	19.86	18.50	5.35	[12.92, 48.80]
LaplaceLatentModel (finetuned)	35.36 M	10.19	9.30	2.93	[6.75, 23.18]
CorrectionAE (+ UNet)	35.84 M	4.80	4.46	1.56	[2.37, 13.28]

Table 1: Quantitative comparison of transient surrogates on the test set. The baseline latent surrogate achieves 19.86% mean error. End-to-end fine-tuning reduces this to 10.19% (48% improvement), while the UNet residual corrector further reduces error to 4.80% (76% improvement over baseline).

[6.75%, 23.18%] to [2.37%, 13.28%], with the tightest standard deviation (1.56%), indicating highly stable and predictable performance.

- **Parameter efficiency.** All three models possess comparable parameter counts ($\sim$ 35M), suggesting that the improvements stem from representation alignment and post-processing rather than architectural complexity. The UNet adds only 481 k trainable parameters for its substantial error reduction.
- **Robustness.** The monotone decrease in standard deviation across the pipeline (5.35% $\rightarrow$ 2.93% $\rightarrow$ 1.56%) demonstrates that each refinement step stabilises predictions, reducing outliers. The corrected model shows no test samples with error exceeding 13.28%, whereas the baseline exhibits errors above 48%.

4 CONCLUSION

This work presents a hierarchy of neural surrogate architectures for PDE emulation centred on learned latent representations. On the stationary problem, a VAE two-stage approach decouples representation learning from parameter regression. On the transient problem, Tucker POD and per-frequency SVD provide linear baselines, while a frequency-conditioned autoencoder with sinusoidal encoding and FiLM modulation learns a non-linear latent space shared across all Laplace frequencies. Per-frequency MLPs map the physical parameters into this latent space; end-to-end fine-tuning then aligns the surrogate with the decoder. A UNet residual corrector applied post hoc further reduces prediction error at marginal cost.

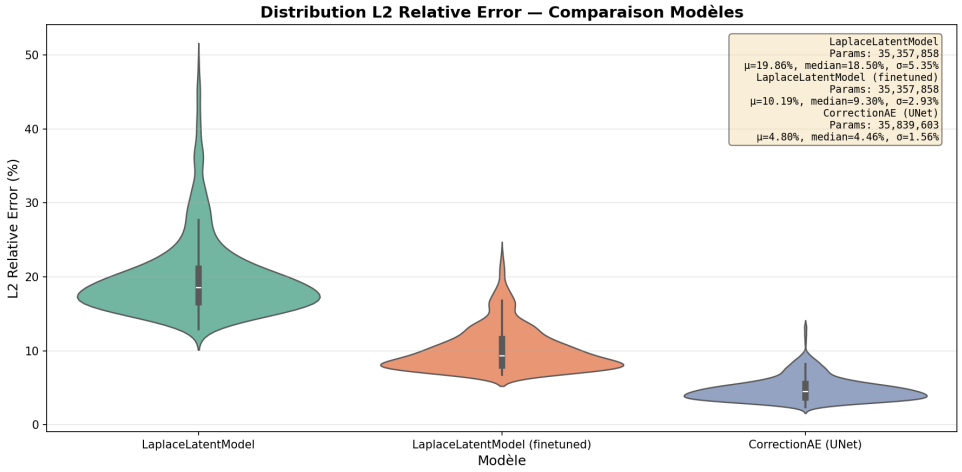


Figure 7: Violin plot comparison of prediction error distributions across all three models. The plot shows the median (horizontal line), quartiles (box), and full distribution (violin shape) for each method.

Future directions include incorporating physics-informed constraints into the latent space, extending the framework to varying source geometries, and applying the residual corrector to other reduced-order surrogate families.

REFERENCES

[1] A. Ammar, M. Ben Saada, E. Cueto, and F. Chinesta. Casting hybrid twin: physics-based reduced order models enriched with data-driven models enabling the highest accuracy in real-time. *International Journal of Material Forming*, 17(2):16, 2024. doi:10.1007/s12289-024-01812-4.

[2] X. Fan, Y. Lin, B. Gong, and H. Li. Offline meteorology-pollution coupling global air pollution forecasting model with bilinear pooling. Preprint, n.d.

[3] Q. Guo, M. Ren, S. Wu, et al. Applications of artificial intelligence in the field of air pollution: a bibliometric analysis. *Frontiers in Public Health*, 10:933665, 2022. doi:10.3389/fpubh.2022.933665.

[4] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021. doi:10.1038/s42254-021-00314-5.

[5] C. Meng, S. Seo, D. Cao, S. Griesemer, and Y. Liu. When physics meets machine learning: a survey of physics-informed machine learning. Preprint, n.d.

[6] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville. FiLM: visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[7] A. Quarteroni, P. Gervasio, and F. Regazzoni. Combining physics-based and data-driven models: advancing the frontiers of research with scientific machine learning. *Mathematical Models and Methods in Applied Sciences*, 35(4):905–1071, 2025. doi:10.1142/S0218202525500125.